

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia Eletrónica Industrial e Computadores

Sistemas Autónomos Inteligentes

Relatório de Cinemática

Autores:

Fábio Alves	PG63877
Lara Fernandes	PG63895
Nádia Silva	PG63900
Nuno Costa	PG63901

Professora:

Estela Bicho

Índice

1 Introdução **1**
1.1 Resumo 1
1.2 Enquadramentos e objetivos 1

2 Formulação da cinemática direta **3**
2.1 Estudo analítico do sistema dinâmico não linear, definido por (1.1) e (1.2): 7
2.2 Estudo analítico do sistema dinâmico linear, definido por (1.11) . . . 7
2.3 Estudo analítico do sistema dinâmico não linear para o comportamento de evitar obstáculos: 9

3 Desenvolvimento/procedimento **11**
3.1 *Autodesk Fusion 360* 11
3.2 *CoppeliaSim* 16
3.3 Matlab 18

4 Resultados e discussão **21**
4.1 Resultados 21
4.2 Discussão 22

5 Conclusão **25**

6 Bibliografia **26**

Lista de Figuras

2.1 Vista lateral do braço no seu plano vertical. 3
2.2 Ângulo segundo o qual o robot "vê", ψ_{target}, o alvo em relação ao sistema dos eixos coordenados do mundo pode ser dado pelo sistema de visão do robot ou ser calculado se as coordenadas do alvo, (X_{target},Y_{target}), forem conhecidas e se tiver. 5
2.3 O sistema perde energia ao longo do tempo. O estado do sistema vai acabar por parar e ficar num estado de repouso final x* (atrator). . . 6
2.4 Representação do comportamento dos sensores do Robot. 8

3.1 Base do robô 11
3.2 Prateleira do robô 12
3.3 Tampa do robô 12
3.4 Suportes do robô 12
3.5 Sensores do robô 13
3.6 Bateria do robô 13
3.7 Motores do robô 14
3.8 Rodas motriz do robô 14
3.9 Rodas Giratórias do robô 14
3.10 Robô 15
3.11 Propriedades dos sensores 16
3.12 Propriedades dos motores 16
3.13 Física das Rodas 17
3.14 Modelo físico do robô 17
3.15 Modelo total do robô 18

4.1 Trajetória para o alvo vermelho (esquerda) e para o verde (direita) . . 21
4.2 Trajetória dos dois robôs para o alvo vermelho (esquerda) e para o verde (direita) 22

1. Introdução

1.1 Resumo

Este projeto consiste no desenvolvimento de um robô autónomo capaz de detetar obstáculos e afastar-se deles, à medida que se aproxima do alvo pretendido. Para a criação deste robô, foi necessário seguir uma determinada ordem de trabalhos para obter o produto final.

Primeiramente, foi necessário o desenvolvimento do modelo do robô, recorrendo-se a uma ferramenta CAD (Autodesk Fusion 360). Nesta plataforma, foram concebidas as diferentes peças que constituem o modelo do robô.

De seguida, todas as peças em CAD foram importadas da plataforma para o CoppeliaSim, de forma a criar o modelo físico do robô.

Por fim, para a navegação autónoma, foram implementados os algoritmos baseados em sistemas dinâmicos não lineares em Matlab e também foi realizada a comunicação entre o Matlab e o CoppeliaSim para o controlo do robô.

1.2 Enquadramentos e objetivos

No âmbito da Unidade Curricular de Sistemas Autónomos Inteligentes do curso de Mestrado em Engenharia Eletrónica Industrial e Computadores da Universidade do Minho, desenvolveu-se um robô autónomo capaz de detetar obstáculos e afastar-se deles, à medida que se aproxima do alvo pretendido. Pretendeu-se desta forma conciliar os conhecimentos adquiridos nas diferentes unidades curriculares da especialização de Controlo, Automação e Robótica e utilizá-los no desenvolvimento do projeto.

O objetivo concreto deste projeto passou, então, por desenvolver um robô desde a sua raiz, criando e/ou escolhendo todas as peças constituintes deste. Este robô tem de ser capaz de se movimentar fazendo uma trajetória completamente autónoma, tendo apenas como conhecimento inicial as coordenadas do alvo final. Durante este trajeto até ao alvo, o robô tem de ser capaz de agir conforme as situações a que poderá vir a ser sujeito, tais como a presença de obstáculos na sua trajetória ou até mesmo alteração em tempo real da posição do alvo.

Tendo isto em conta, para o desenvolvimento do robô é possível identificar os seguintes tópicos:

- Esboço e definição de medidas das várias peças do robô;
- Escolha e pesquisa das peças que constituem o robô;
- Desenvolvimento do modelo do robô com uso da ferramenta CAD;

  - Criação do modelo físico da plataforma em CoppeliaSim;
  - Adaptação dos scripts Matlab para controlo do novo robô;

- Navegação autónoma:
  - -Implementação de algoritmos para a navegação autónoma baseados em sistemas dinâmicos não-lineares;
  - Desenvolvimento de um programa em Matlab e comunicação entre Matlab e CoppeliaSim para o controlo do robô;

2. Formulação da cinemática direta

A cinemática direta permita obter a posição da garra (x, y, z) apartir do ângulo das juntas $(J1, \ldots, J6)$. A solução é única e sempre existente.

Cada junta do robô apresenta diferentes comportamentos, logo teremos que alterar a abordagem para cada junta:

- A junta 1 é responsável pela rotação do braço, em relação ao eixo vertical, logo define o plano vertical no qual o braço se movimenta.;
- As juntas 2 e 3 comportam-se como um braço de 2 DOFs, logo é possível aplicar uma abordagem geométrica para obter a posição de J4 *(Pulso)*;
- A junta 4 é responsável pela orintação de L4 e da garra, logo a partir da posição de J4 é possível obter a posição de J5;
- A junta 5 é fixada a 0 graus, para a garra se manter no plano do braço;
- A junta 6 é responsável pela rotação da garra *(Yaw)*, logo não afeta a posição da garra.

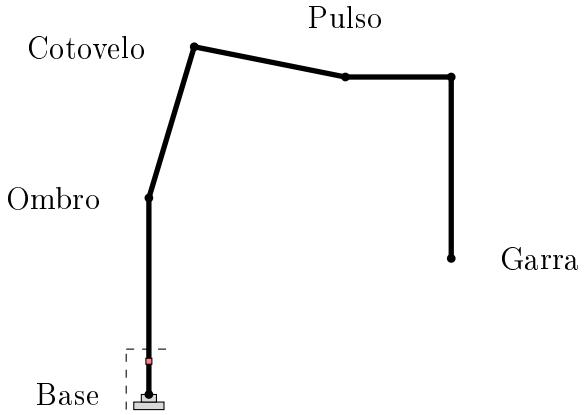


Figura 2.1: Vista lateral do braço no seu plano vertical.

Navegação autónoma, exige a criação de algoritmos de modo a tornar possíveis vários fatores essenciais no seu processo, tais como:

- Navegação na direção de uma posição alvo (*target acquisition*);
- Evitar colisões com obstáculos (*obstacle avoidance*);

- Integração do comportamento de navegação na direção de uma posição alvo e evitar colisões com obstáculos (dinâmica não-linear);
- Integração do comportamento de navegação na direção de uma posição alvo e evitar colisões com obstáculos (dinâmica linear).

Em respeito ao *Target acquisition*, um robô móvel autónomo para ser útil tem que ser capaz de navegar para uma posição alvo. A informação sobre o alvo pode ser definida pelas coordenadas do alvo (x_{alvo}, y_{alvo}) ou o sistema de visão pode indicar em que direção está a detetar o alvo (Figura 2.2).

Dinâmica de navegação:

$$\theta_A = q_2 + \frac{\pi}{2}, \tag{2.1}$$

Equação (2.1)

Onde,

$$\frac{a^2 + b^2}{c^2} = 1 \tag{2.2}$$

Equação (2.2)

Dinâmica da velocidade linear:

$$\frac{dv}{dt} = g_{target}\left(v\right)$$

Equação (1.3)

Através do estudo analítico dos sistemas dinâmicos é possível a atribuição de valores adequados aos seus parâmetros por forma a que os atractores surjam com os devidos valores, e que simultaneamente as variáveis de estado consigam seguir de perto um atractor à medida que este se desloca. Contribuindo assim para a estabilidade assimptótica do sistema que gera o comportamento do robô e o torna mais robusto face a perturbações externas.

O método de *Lyapunov* é um método que consiste em encontrar uma função U(x) continuamente diferenciável, chamada função de *Lyaponov*, representando a 'energia' do sistema. No qual, através desta função é possível investigar a estabilidade dos pontos fixos de um sistema dinâmico.

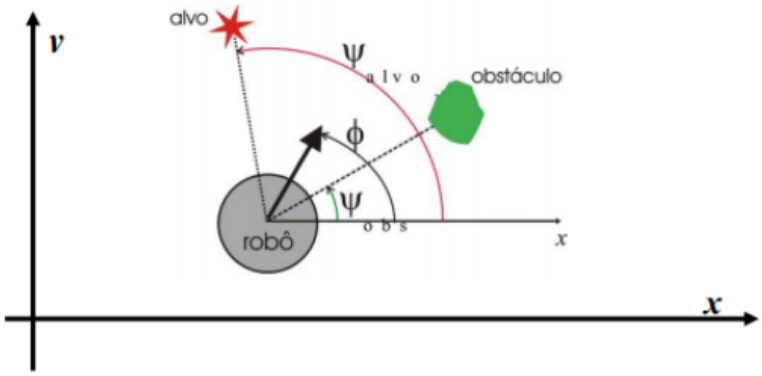


Figura 2.2: Ângulo segundo o qual o robot "vê", ψ_{target}, o alvo em relação ao sistema dos eixos coordenados do mundo pode ser dado pelo sistema de visão do robot ou ser calculado se as coordenadas do alvo, (X_{target},Y_{target}), forem conhecidas e se tiver.

Considerando o conceito de 'energia' e olhando para o sistema dinâmico como se fosse um sistema físico dissipativo, isto é, sendo uma função que a cada estado do sistema x associa uma quantidade de energia potencial U(x). Se o sistema for tal que perde energia ao longo do tempo, e a energia perdida nunca é restabelecida (figura 2.3) verifica-se que

$$\frac{dU(x)}{dt} < 0$$

Equação (1.4)

e que eventualmente o estado do sistema vai acabar por parar e ficar num estado de repouso final x*(atrator). Neste, se o estado do sistema deixa de variar, então também a sua energia deixa de variar, pelo que se verifica

$$\frac{dU(x)}{dt} = 0$$

Equação (1.5)

Tendo em consideração agora o `conceito de sistema dinâmico gradiente`, i.e. um sistema cujo campo vetorial resulta do gradiente de uma função:

ou seja

Cruzando ambos os conceitos obtém-se que

Ou seja, ao longo do tempo a energia U(t) diminui ao longo de toda a trajetória x(t), por conseguinte a evolução do sistema é sempre no sentido de evoluir para um estado de menor energia.

Adicionalmente, repare-se que

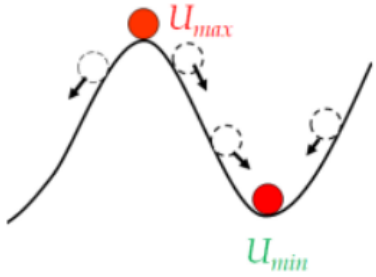


Figura 2.3: O sistema perde energia ao longo do tempo. O estado do sistema vai acabar por parar e ficar num estado de repouso final x* (atrator).

$$\frac{dx}{dt} = -\frac{dU(x)}{dx}$$

$$\frac{dx}{dt} = -\frac{dU(x)}{dx}$$

Equação (1.6)

$$\frac{dU(x)}{dt} = \frac{dU(x)}{dx}\frac{dx}{dt} = -\left(\frac{dU(x)}{dx}\right)^2 \leq 0$$

Equação (1.7)

$$\frac{dU(x)}{dt} = \frac{dU(x)}{dx}\frac{dx}{dt} = -\left(\frac{dU(x)}{dx}\right)^2 = -(f(x))^2$$

Equação (1.8)

Assim, se o sistema for colocado nos pontos de equilíbrio ou pontos fixos x* então a energia do sistema não varia,

$$\frac{dU(x)}{dx} = 0 \qquad \text{e} \qquad U = constante$$

Equação (1.9)

Isto porque, nos pontos fixos x* temos

$$\frac{dU(x)}{dt}|_{x=x^*} = f(x^*) = 0$$

Equação (1.10)

2.1 Estudo analítico do sistema dinâmico não linear, definido por (1.1) e (1.2):

Sabidos os pontos físicos do sistema dinâmico (1.1) admitindo que f_{target} é definido por (1.3),

$$\Phi_1^* = \lambda_{target}$$

$$\Phi_2^* = \pi + \lambda_{target}$$

A sua estabilidade é classificada como:

Para Φ_1^*: $\lambda_{target} > 0 \rightarrow$ Assintoticamente estável (atractor)

$\lambda_{target} < 0 \rightarrow$ Instável (repulsor)

Para Φ_2^*: $\lambda_{target} > 0 \rightarrow$ Instável (repulsor)

$\lambda_{target} < 0 \rightarrow$ Assintoticamente estável (atractor)

Logo, para o robô ter o comportamento desejado, uma vez que o alvo funciona como um ponto fixo atractor e o robô desloca-se para o alvo,

$\lambda_{\text{target}} > 0$

Sabendo também que a constante de tempo $\tau = 1/\lambda_{\text{target}}$,

$\uparrow \lambda_{\text{target}} \rightarrow \downarrow \tau$

Mais rapidamente converge o valor de ϕ para o atrator.

2.2 Estudo analítico do sistema dinâmico linear, definido por (1.11)

$$\frac{d\phi}{dt} = f_{target}(\Phi) = -\lambda_{target}(\Phi - \Psi_{target})$$

Equação (1.11)

Sabidos os pontos físicos do sistema dinâmico,

$\phi* = \psi_{\text{target}}$

A sua estabilidade é classificada como: Em respeito ao *obstacle avoidance*, um

Para Φ^*: $\lambda_{target} > 0 \rightarrow$ Assintoticamente estável (atractor)

$\lambda_{target} < 0 \rightarrow$ Instável (repulsor)

robot móvel autónomo para ser capaz evitar obstáculos recorre a sensores que sejam capazes de detetar as obstruções que aparecem nas suas direções. Detetados os obstáculos, os sensores permitem também estimar a distância a que o robot se encontra destes (figura 2.4). O controlo do movimento do robot na direção do alvo é realizado através de sistemas dinâmicos para a direção de navegação e velocidade linear.

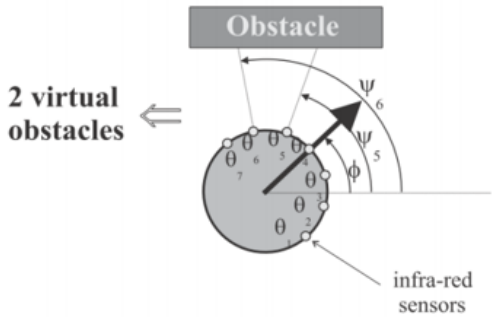


Figura 2.4: Representação do comportamento dos sensores do Robot.

Dinâmica de navegação:

$$\frac{d\Phi}{dt} = F_{obs}(\Phi) = \sum_i f_{obs,i}(\Phi) + f_{stoch}, \quad [0,2\pi] \bmod 2\pi$$

Equação (2.1)

Onde,

$$f_{obs}(\Phi) = \lambda_{obs,i}(\Phi - \Psi_{obs,i}) e^{-\frac{(\Phi - \Psi_{obs,i})^2}{2\sigma_i^2}} \qquad \sigma_i = \arctan\left(\tan\left(\frac{\Delta\theta}{2}\right) + \frac{R_{robot}}{d_i + R_{robot}}\right)$$

Equação (2.2) e (2.3)

Uma vez mais, através do estudo analítico dos sistemas dinâmicos é possível a atribuição de valores adequados aos seus parâmetros por forma a que, neste caso, os repulsores surjam com os devidos valores.

$$\lambda_{obs,i} = \beta_1 e^{\frac{-d_i}{\beta_2}} \qquad f_{stoch=\sqrt{Q}x_n(t)}$$

Equações (2.4) e (2.5)

2.3 Estudo analítico do sistema dinâmico não linear para o comportamento de evitar obstáculos:

Sabidos os pontos físicos do sistema dinâmico (2.1) quando apenas 1 obstrução, $f_{obs,i}$, é detetada

$\phi = \psi_{obs,i}$

A sua estabilidade é classificada como:

Para Φ^*: $\Psi_{obs,i} > 0 \rightarrow$ Instável (repulsor)

$\Psi_{obs,i} < 0 \rightarrow$ Assintoticamente estável (atractor)

Sabendo também que a constante de tempo $\tau = 1/\psi_{obs,i}$,

$\downarrow \psi_{obs,i} \rightarrow \uparrow \lambda_{target} \rightarrow \uparrow \tau$

Mais rapidamente converge o sistema para o ponto fixo.

Logo, uma vez que o objetivo é o robô desviar-se dos obstáculos, para o robô ter o comportamento desejado o ponto de equilíbrio tem de ser um repulsor.

$$\lambda_{obs,i} = \beta_1 e^{\frac{-d_i}{\beta_2}} > 0$$

Onde λ(obs,i) depende da distância (di) e do parâmetro β_1,

$$\beta_1 = \frac{\lambda_{obs,i}}{e^{\frac{-d_i}{\beta_2}}} > 0$$

Para se calcular o valor máximo de β_1 temos de calcular λ_{obs_max},

$$d_i = 0 \rightarrow \lambda_{obs_max} = \beta_1$$

Concluindo que para que o robot tenha o comportamento desejado, o parâmetro β1 tem de assumir o valor de,

$$\beta_1 = \frac{1}{\tau_{obs_min}} \text{ e } \beta_1 = \frac{\lambda_{obs,i}}{\frac{-d_i}{e^{\beta_2}}} > 0.$$

Em respeito à integração do comportamento de navegação na direção de uma posição alvo e evitar colisões com obstáculos (dinâmica não-linear e dinâmica linear), consiste na integração dos dois fatores anteriores, o target *acquistion* e o *obstacle avoidance*.

Matlab, é uma excelente plataforma de programação para o desenvolvimento de algoritmos e criação de modelos.

Através da alteração de *scripts Matlab* é possível a ligação a ambientes de simulação e modelos criados em *CoppeliaSim*. Sendo estas alterações realizadas no construtor de classes, onde são inicializados os *handles*.

3. Desenvolvimento/procedimento

Para o desenvolvimento do robô, foi necessário desenvolver o modelo *CAD*, a criação do modelo físico em *CoppeliaSim*, a implementação de código em *Matlab* e ainda a comunicação entre o *CoppeliaSim* e *Matlab*. Relativamente à parte de desenvolvimento do modelo *CAD* do robô, conceberam-se diversas peças desde a sua raiz e foram ajustadas peças já concebidas. Por outro lado, adaptou-se código para controlo do robô, nomeadamente para se desviar de obstáculos e atingir um alvo. De modo a ser possível controlar o robô foram ajustados diversos parâmetros posteriormente testados.

3.1 *Autodesk Fusion 360*

No desenvolvimento do modelo CAD do robô foram concebidas diversas peças: uma base, uma prateleira, uma tampa, uma bateria e quatro suportes. Para além destas peças, foram escolhidas peças já desenvolvidas: duas rodas motrizes, duas rodas giratórias, dois motores e doze sensores. Após todas as peças desenvolvidas e ajustadas, foi feito o assembler das diversas peças formando assim o robô. De seguida, é possível visualizar os desenhos técnicos de cada peça, a peça em modelo CAD e uma breve descrição de cada peça:

- *Base:* Projetou-se uma base cilíndrica com 1cm de altura e 15 cm de raio. Esta base serve de suporte a todo o robô e têm um encaixe para os motores (Figura 3.1).

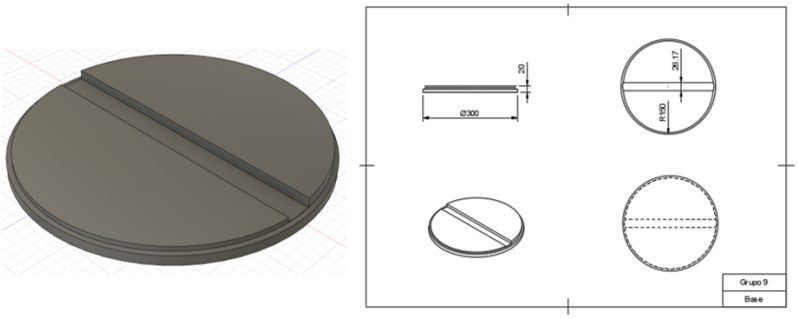


Figura 3.1: Base do robô

- *Prateleira:* Foi desenvolvida uma prateleira com 9 cm de altura, para criar duas divisões na base. Sendo a divisão inferior para a colocação dos motores e fios necessários e na divisão superior é colocada bateria. Esta prateleira serve para isolar todos os elementos que estão apoiados na base do exterior conferindo uma maior segurança dos mesmos (Figura 3.2).

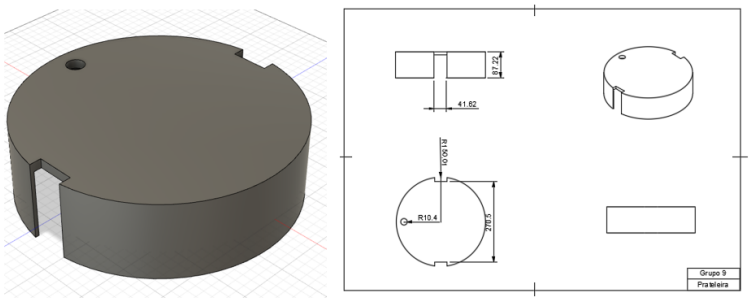


Figura 3.2: Prateleira do robô

- *Tampa:* Elaborou-se uma tampa com 2cm de altura e um raio de 15 cm, para que esta fique apoiada nos suportes. Esta tampa é a camada mais alta do robô e protege os elementos que estão abaixo dela (Figura 3.3).

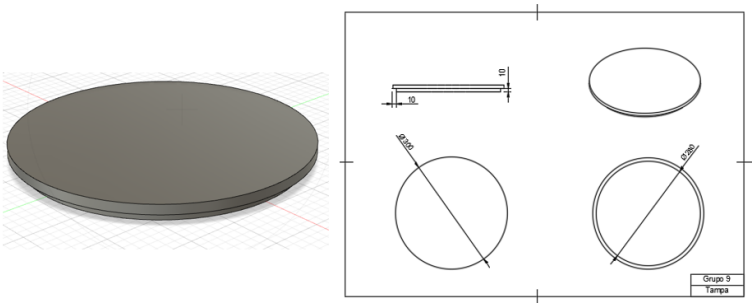


Figura 3.3: Tampa do robô

- *Suportes:* Para dar altura ao robô de forma a colocar alguns elementos fora do interior do robô foram criados 4 suportes (Figura 3.4).

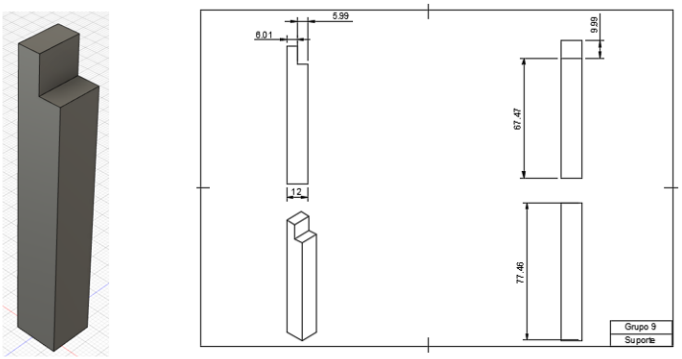


Figura 3.4: Suportes do robô

- *Sensores:* O modelo do robô possui 12 sensores infravermelhos previamente concebidos, nos quais foi necessário ajustar as medidas. Estes estão igualmente espaçados entre si ao redor de toda a base. Estes sensores têm um alcance de 80 cm e uma amplitude de 30° (Figura 3.5).

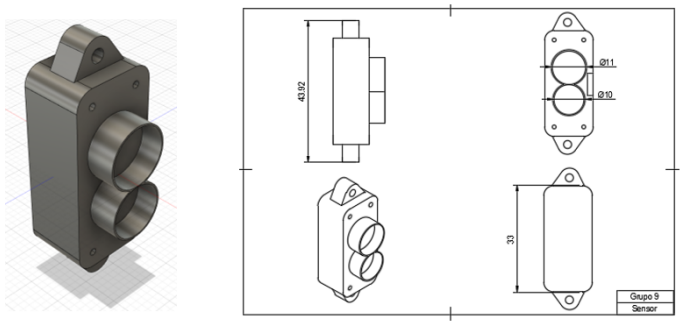


Figura 3.5: Sensores do robô

- *Bateria:* : Foi desenvolvida uma bateria com 15 cm de comprimento, 10,5 cm de altura e 6,5 cm de largura. A bateria foi desenhada com base em modelos reais. Para fazer a bateria, foi desenhado um retângulo com 15 x 6,5 cm, utilizando a ferramenta *Create Sketch*, de seguida, através da ferramenta extrude foi dada uma altura de 10,5 cm, conferindo-lhe assim a forma de paralelepípedo. Após isto feito, foi desenho um novo retângulo no topo do paralelepípedo e novamente através da ferramenta extrude foi removido o centro do retângulo de forma a criar os terminais que se encontram na parte de cima da bateria como é vísivel na figura seguinte (Figura 3.6).

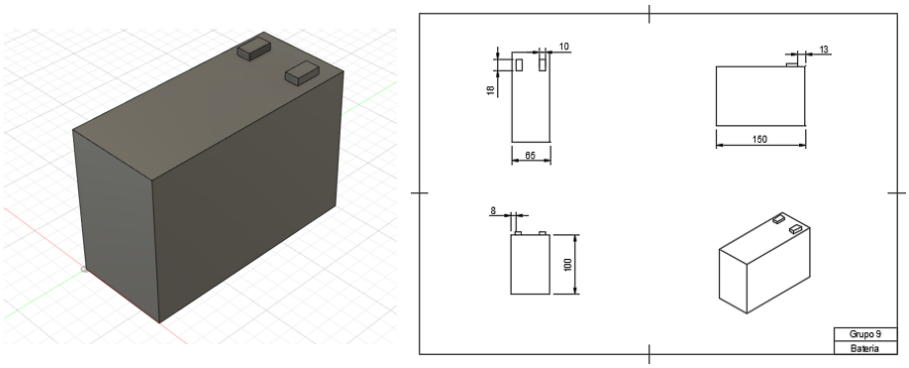


Figura 3.6: Bateria do robô

- *Motor:* O modelo do robô possui dois motores 12V DC. Foi utilizado um design já concebido, sendo depois necessário adaptar às características do nosso robô (Figura 3.7).

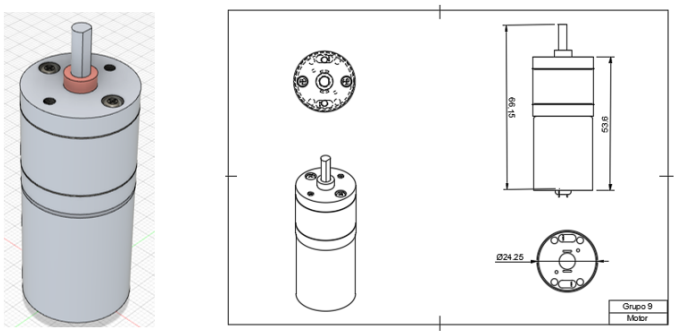


Figura 3.7: Motores do robô

- *Rodas motriz e rodas giratórias:* O modelo do robô possui duas rodas motrizes e duas rodas giratórias. Foram utilizados designs já concebidos, sendo depois necessário adaptar às características do nosso robô (Figura 3.8 e Figura 3.9).

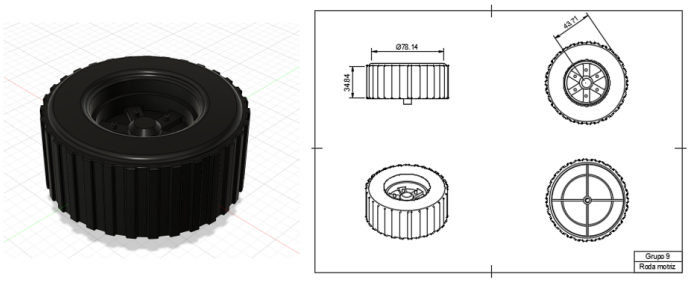


Figura 3.8: Rodas motriz do robô

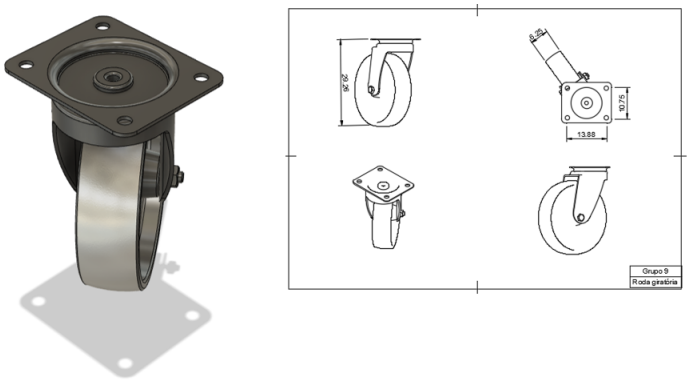


Figura 3.9: Rodas Giratórias do robô

- *Robô:* O robô é a junção de todas as peças mencionadas anteriormente (Figura 3.10).

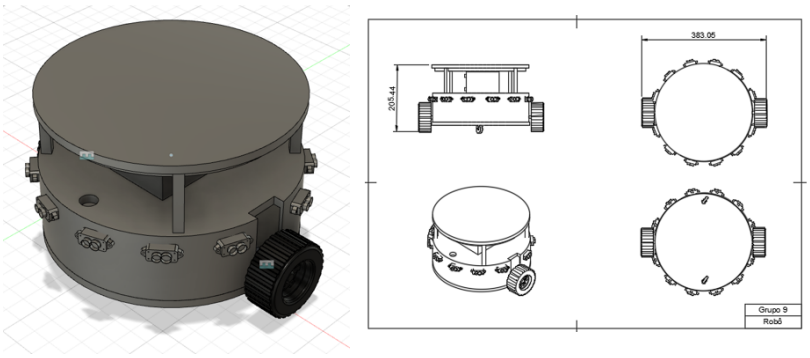


Figura 3.10: Robô

3.2 *CoppeliaSim*

O *CoppeliaSim* é um simulador de robôs que usa um mecanismo de cinemática para cálculos de cinemática direta e inversa e várias bibliotecas de simulação. Modelos e cenários são construídos pela montagem de vários objetos numa estrutura hierárquica.

Após ter sido feito o modelo *CAD*, há duas etapas para a criação de um projeto em *CoppeliaSim*: criar um modelo físico e adaptar os scripts *Matlab* para controlo do robô. Posto isto, desenvolveu-se um modelo físico simples para testar a física do robô, como é possível observar na Figura 3.14.

Para o desenvolvimento deste modelo, é necessário criar uma forma primitiva. Neste caso foi utilizada a forma de cilindro para desenvolver o corpo do robô e as duas rodas. De seguida é necessário adicionar motores e sensores. Para a criação e ajuste da localização de cada peça foram utilizadas as ferramentas *Object/item shift* e *Scene Object Properties*. Sendo que se trata de um modelo físico, foi adicionada física, através da ferramenta *Show dynamic properties dialog* em *Scene Object Properties* às rodas do modelo, atribuídos valores aos motores e através da ferramenta *Show volume parameters*, foram configurados todos os sensores.

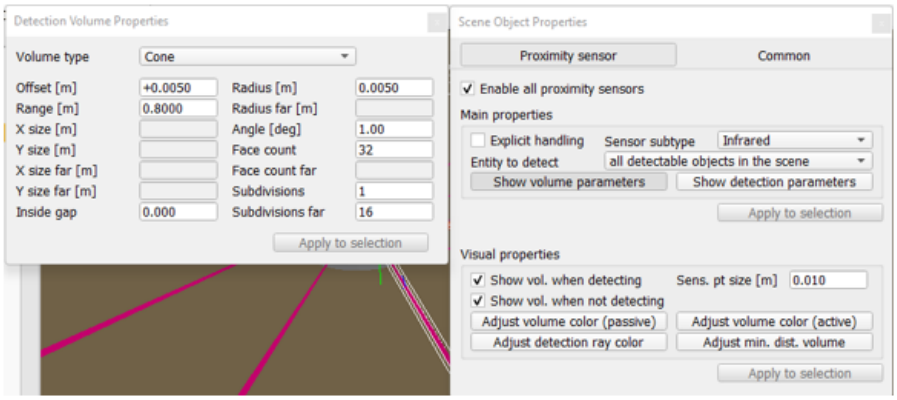


Figura 3.11: Propriedades dos sensores

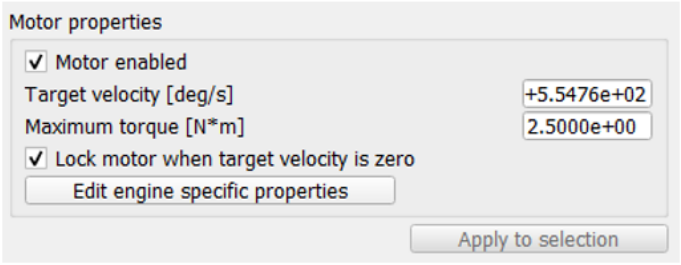


Figura 3.12: Propriedades dos motores

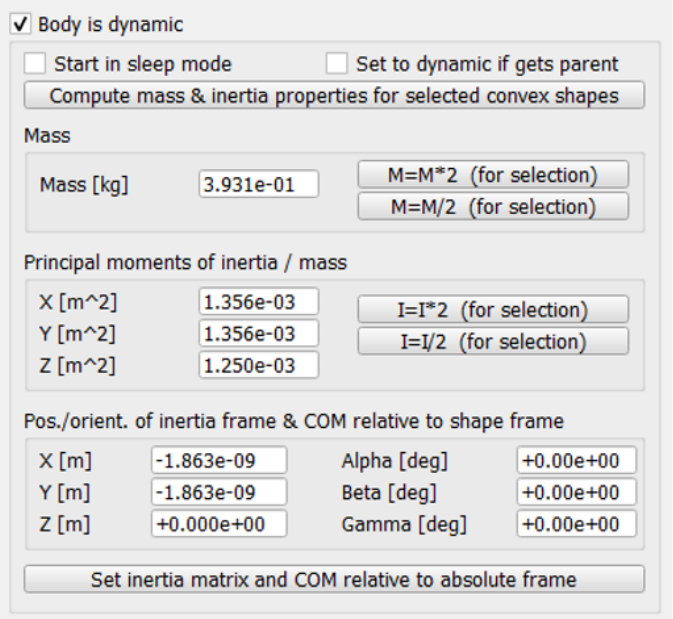


Figura 3.13: Física das Rodas

Depois de testado o modelo físico, foi exportado o robô desenvolvido no *Fusion* em formato *.obj*, e inseriu-se este modelo no *CoppeliaSim* sendo usado apenas como modelo gráfico, ou seja, serve apenas para visualizar. Isto pode ser feito através da ferramenta *Common- Visibility* também em *Scene Object Properties* . De seguida, ao robô obtido no *CoppeliaSim*, adicionaram-se os sensores e foram colocados os elementos do robô de forma hierárquica para respeitar a física e comportar-se de acordo com o objetivo. Além disso foram utilizados cenários já desenvolvidos onde se colocou o robô e procederam-se testes para diversas situações.

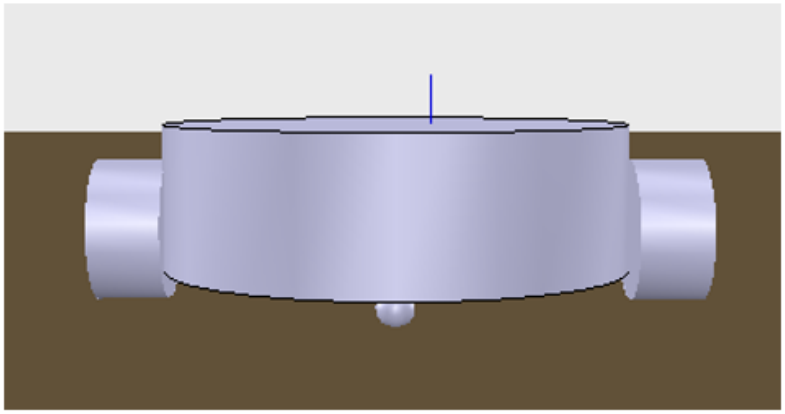


Figura 3.14: Modelo físico do robô

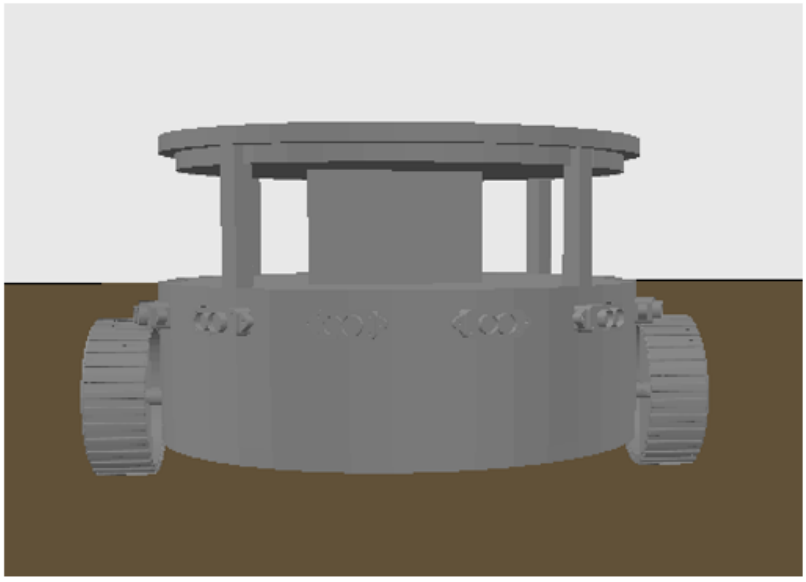


Figura 3.15: Modelo total do robô

3.3 Matlab

Por último, recorrendo ao código utilizado na unidade curricular de Sistemas Dinâmicos Não Lineares, foram efetuados alguns ajustes e alterações de acordo com as propriedades do robô desenvolvido.

No ficheiro program_dyn_EB_tar_obs, foram ajustados os parâmetros da velocidade do robô, taxa de decaimento, magnitude de repulsão e a magnitude do ruído, bem como foram acrescentadas/alteradas algumas partes do código. Uma vez que foi desenvolvido o controlo autónomo de dois robôs, ainda neste ficheiro, foram duplicadas as linhas de código de forma a controlar tanto o veículo 1 como o 2. De forma a ser possível visualizar graficamente dados relativos ao movimento do robô foram acrescentadas linhas de código com os plots. Todas estas alterações estão visíveis nas figuras seguintes.

Relativamente ao ficheiro RobDyn_robot, alteraram-se as dimensões do robô e o número de sensores aplicados, de forma a corresponder ao robô desenvolvido. Para além disto, este ficheiro foi duplicado, RobDyn_robot2, tendo sido nele alterados nomes de variáveis e funções uma vez que com a utilização de dois robôs, é necessário que estes tenham variáveis e funções com nomes distintos. Para além destes ficheiros, foi utilizado o ficheiro RobDyn_simulation para tornar possível a conexão dos dois robôs no *CoppeliaSim*.

De modo a testar o funcionamento do robô e validar a navegação autónoma foi testado um cenário variando algumas características do mesmo.

```
        function [obj,error] = RobDyn_robot(sim_obj)

            error = 0;

            obj.TARGET_Number=2;

            obj.Rrobot = 48.0/2;

            obj.Dwheels = 0.32;    %m

            obj.RadiusWheels = 0.1/2;    %m

            obj.NsensorsIRobs = 12;

            [obj.vrep, obj.clientID] = sim_obj.get_connection();
```

Listing 3.1: Objeto RobDyn_robot

```
        function [obj,error] = RobDyn_robot2(sim_obj)

            error = 0;

            obj.TARGET_Number=2;

            obj.Rrobot = 48.0/2;

            obj.Dwheels = 0.32;    %m

            obj.RadiusWheels = 0.1/2;    %m

            obj.NsensorsIRobs = 12;

           [obj.vrep, obj.clientID] = sim_obj.get_connection();
```

Listing 3.2: Objeto RobDyn_robot2

```
[vehicle2,error2] = RobDyn_robot2(sim);
[vehicle,error] = RobDyn_robot(sim);
```

Listing 3.3: Instanciação dos veículos

```
   %%----------- BEGIN YOUR CODE HERE ----------- %
   %Set parameter values
   Tau_tar=10*timestep;
   lambda_tar=1/Tau_tar;
   N=length(dc);
   dtheta_obs=theta_obs(2)-theta_obs(1);
%    dtheta_obs= 2*pi/N ;%gama angular dos setores
   beta1=22.5;
   beta2=20;
   too_far=80;

   %Create vectores for psi_obs,lambda_obs, sigma_obs
   Q=0.01; %magnitude stochastic force

   psi_obs=zeros(N,1);
   lambda_obs=zeros(N,1);
   sigma_obs=zeros(N,1);
   V_pot=zeros(N,1);

   %compute the contribution
   f_obs=zeros(N,1);
```

```
22      F_obs=0.0;
23      U = 0;
24
25  %     for i=2:1:(N-1)
26      for i=1:N
27      % compute repeller value
28         psi_obs(i)=  theta_obs(i)+phirobot;
29
30         %compute magnitude of repulsion
31         if(dc(i) < too_far)
32           lambda_obs(i)=beta1*exp(-dc(i)/beta2);
33  %        else
34  %            lambda_obs(i)=0;
35         end
36         %compute of range of repulsion
37         sigma_obs(i)=atan(tan(dtheta_obs/2)+Rrobot/(Rrobot+dc(i)));
38
39         %compute f_obs_i
40         f_obs(i)=lambda_obs(i)*(phirobot-psi_obs(i))*exp(-(phirobot-psi_obs(i))^2/(2*(
       sigma_obs(i)^2)));
41
42  %        V_pot(i)= lambda_obs(i)*(sigma_obs(i)^2)*exp(-(((phirobot-psi_obs(i))^2)/(2*(
       sigma_obs(i))^2)))-(lambda_obs(i)*((sigma_obs(i)^2)/sqrt(exp(1))))*0.1;
43         V_pot(i)= lambda_obs(i)*(sigma_obs(i)^2)*exp(-(((phirobot-psi_obs(i))^2)/(2*(
       sigma_obs(i))^2)))-lambda_obs(i)*sigma_obs(i)^2/sqrt(exp(1))*0.05;
44         U = U + V_pot(i);
45         F_obs=F_obs+f_obs(i);%+V_pot(i);
46
47      end
48
49
50      %compute psi_tar
51      psi_tar=atan2((YTARGET-yrobot),(XTARGET-xrobot));
52      %erro=phirobot-psi_tar;
53  %     erro=phirobot-psi_tar;
54
55      %compute ftar
56      ftar=-lambda_tar*sin(phirobot-psi_tar);
57
58      %compute stochastic force
59      fstoch=sqrt(Q)*randn(1,1);
60
61      %compute total force
62      f_total=ftar+F_obs+fstoch;
63
64      %compute angular velocity
65      wrobot=f_total;
66
67      distance = sqrt((xrobot-XTARGET)^2+(yrobot-YTARGET)^2);
68      %set linear velocity
69
70      alpha = atan(2*U)/pi;
71
72      vrobot = 50*(1-alpha);
73
74      if(distance < 3.5*Rrobot)
75          if (ntarget == 1)
76              ntarget = 2;
77          else
78              ntarget = 1;
79          end
80      end
```

Listing 3.4: Código de navegação do veículo 1

4. Resultados e discussão

Neste subcapítulo, são apresentados os resultados obtidos após a junção do modelo físico do robô, elaborado no CoppeliaSim, com as alterações efetuadas no código Matlab.

4.1 Resultados

Para isso, realizaram-se vários testes de forma a obter-se o desempenho pretendido, ou seja, um robô capaz de navegar autonomamente até a um ou mais alvos num cenário que contém obstáculos. Nas figuras seguintes pode-se comprovar que se alcançou o comportamento desejado.

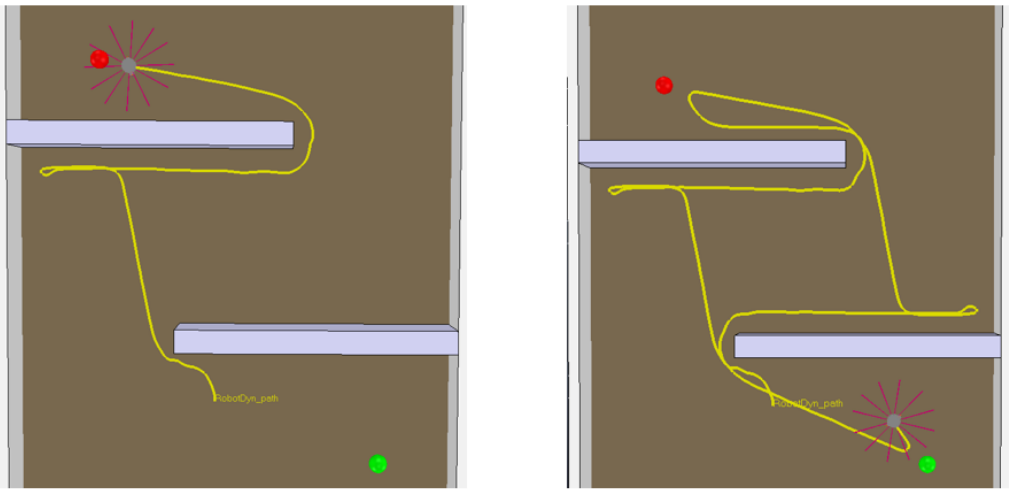


Figura 4.1: Trajetória para o alvo vermelho (esquerda) e para o verde (direita)

O robô tinha como destino inicial o alvo representado pela esfera vermelha. Para lá chegar deparou-se com dois obstáculos. Quando um obstáculo é detetado, a velocidade do robô diminui de modo que este tenha uma resposta eficaz no desvio do obstáculo, evitando a colisão com o mesmo. Sendo assim, na figura da esquerda, confirma-se que o robô se desviou com sucesso, conseguindo, por fim, alcançar o alvo. Após esse momento, o robô encontrou-se pronto para seguir para o próximo alvo, representado pela esfera de cor verde, onde mais uma vez teve de contornar os obstáculos detetados pelos sensores, e alterar a velocidade, conforme a situação em que se encontrava, durante o seu percurso até chegar ao alvo pretendido, como se vê na figura da direita. Também na figura da direita, pode-se observar toda esta trajetória

que o robô desempenhou sendo que, neste caso, o seu destino mudava de um alvo para o outro sempre que um deles era alcançado.

Obtido o desempenho pretendido para o projeto, prosseguiu-se com a adição extra de um robô. O objetivo é que este siga o robô principal pois, para o novo robô, esse é o seu alvo. Sendo assim, o segundo robô tem que acompanhar a trajetória que o primeiro robô executa em direção a um dos alvos. Os resultados obtidos podem ser vistos nas figuras seguintes.

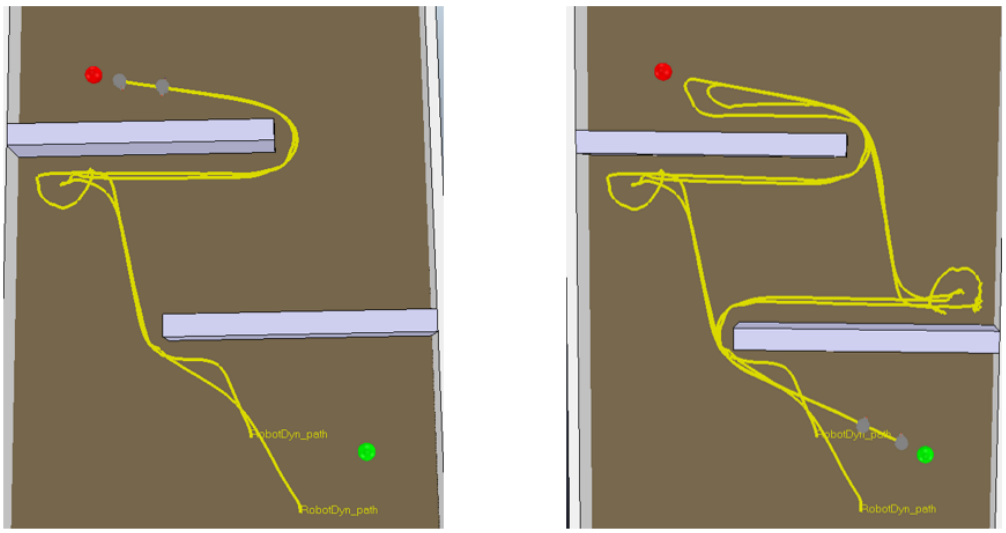


Figura 4.2: Trajetória dos dois robôs para o alvo vermelho (esquerda) e para o verde (direita)

Tal como no caso anterior, o destino inicial era o target vermelho e tanto o robô principal como o secundário tiveram que se desviar dos dois obstáculos para chegarem ao destino. O procedimento para o fazerem foi o mesmo que o do teste com apenas um robô mencionado em cima. Ainda é de referir que o segundo robô também deteta os obstáculos e a velocidade dos dois robôs também se ajusta perante a situação em que se encontram. O segundo robô mantém sempre uma distância de 60 cm do primeiro robô para evitar colisões entre eles. Com estas condições, o primeiro robô quando encontra um obstáculo e uma parede lateral tem mais dificuldade em sair desse local visto que o segundo robô também é considerado um obstáculo para o primeiro. No entanto, consegue desviar-se e seguir a sua rota para qualquer um dos targets, vermelho ou verde, tal como no caso anterior.

4.2 Discussão

Obtidos os resultados, far-se-á, neste capítulo, uma análise dos mesmos de forma a entender o comportamento do robô em cada situação.

Começando pelo cenário em forma de S e com apenas um robô, quando o robô deteta um obstáculo, a força repulsiva do obstáculo torna-se cada vez mais forte à medida que mais sensores detetam obstáculos virtuais e sobrepõe-se à força atrativa do alvo. Dá-se, nesse momento, uma bifurcação onde o ponto fixo atrator que atraía o robô na direção do alvo, altera-se para um repulsor, logo o robô responde, afastando-se do obstáculo. Como de cada lado do repulsor existe um atrator, há duas possibilidades, virar à esquerda ou virar à direita. A escolha entre essas duas possibilidades depende da bacia de atração em que a direção de navegação fica no momento da bifurcação.

À medida que o robô se move, a informação sensorial varia e, por conseguinte, os atratores e os repulsores deslocam-se. Sendo assim, o controlo da velocidade é fulcral para manter o sistema estável, ou seja, num, ou perto de um, atrator. Para isso, a velocidade de translação do robot deve ser controlada de forma que a velocidade com que os atractores se deslocam seja inferior à velocidade de relaxamento da dinâmica da direção de navegação.

Quando nenhuma obstrução é detetada, a velocidade do robô é estabilizada por um atrator cujo valor é proporcional à distância ao alvo, de acordo com a contribuição do alvo imposta pela função potencial. Quando são detetadas obstruções, o valor de intensidade da força de atração do obstáculo é positivo para o valor corrente da direção de navegação, e neste caso, a dinâmica de velocidade é governada por um atrator que é especificado pelas contribuições dos obstáculos. Todavia, quando são detetadas obstruções, mas a direção de navegação do robô fica fora da zona repulsiva criada pelas obstruções, a dinâmica da velocidade é dominada pela contribuição do alvo e o robô acelera novamente.

Para analisar se as contribuições dos obstáculos estão presentes, integram-se as forças de repulsão dos obstáculos o que resulta numa função potencial do sistema dinâmico que gera o comportamento "evitar colisões com obstáculos". Valores positivos desta função indicam que a direção de navegação está dentro duma zona repulsiva, criada pelos obstáculos, pelo que é necessário ter a intensidade de atração do obstáculo maior que 0 e a intensidade de atração do alvo igual a 0. Reciprocamente, valores negativos do potencial indicam que a direção de navegação está fora da zona repulsiva ou então que as forças repulsivas dos obstáculos são fracas, daí que a intensidade de atração do obstáculo igual a 0 e a intensidade de atração do alvo maior que 0 seja necessário neste caso. A seguinte hierarquia de taxas de relaxação assegura que o todo o sistema converge para os atractores e que o comportamento "evitar obstáculos" tem prioridade sobre o comportamento "movimento na direção do alvo":

$$\lambda_{\text{alvo}} \ll c_{v,\text{alvo}}, \qquad \lambda_{\text{obs}} \ll c_{v,\text{obs}}, \qquad \lambda_{\text{alvo}} \ll \lambda_{\text{obs}}$$

Os testes feitos, para o caso de apenas um robô, foram acompanhados de gráficos dinâmicos que se iam alterando conforme a situação em que o robô se encontrava.

Observa-se nesses gráficos a força atrativa do alvo, a dinâmica resultante das contribuições do alvo e obstáculos e, por último, a força dos obstáculos e a função potencial. Através das suas análises, confirma-se que o gráfico da força atrativa do alvo mostra, com o declive negativo, o ponto fixo atrator que atrai o sistema para o valor que ele especifica e um ponto fixo repulsor com o declive positivo na direção oposta à direção onde se localiza o alvo. Verifica-se, também, que a força repulsiva dos obstáculos aumenta quando mais sensores detetam o obstáculo e a distância diminui. Quanto ao gráfico da dinâmica total, quando já não é necessário que o robô se desvie de um obstáculo, a dinâmica da direção de navegação forma um atractor na direção que aponta para o alvo. Inversamente, a dinâmica da direção de navegação edifica um repulsor nessa direção quando se tem que desviar do obstáculo. Finalmente, a função potencial resulta do integral da força repulsiva que é máxima perto da direção onde se localiza o obstáculo que tem que ser evitada. Quanto maior a força repulsiva, menor é a velocidade de translação do robô.

Quanto ao cenário em forma de S mas com os dois robôs, o comportamento do primeiro robô é justificado da mesma forma que o caso anterior. Já o segundo robô tem o robô que está à sua frente como o seu alvo, ou seja, um atrator, pelo que o seu comportamento altera-se conforme a situação que o primeiro robô enfrenta. No caso do robô secundário estar a detetar obstáculos, as forças repulsivas predominam em relação à força atrativa do robô primário. Logo, deixa de seguir o primeiro robô e desvia-se do obstáculo seguindo a justificação dada anteriormente. Após ultrapassar o obstáculo, volta a seguir o robô.

5. Conclusão

Através da realização deste projeto no âmbito da Unidade Curricular Sistemas Autónomos Inteligentes, desenvolveram-se técnicas e competências aprendidas durante o percurso universitário e conseguiu-se também entender de uma forma mais explícita e concreta como funciona o desenvolvimento de um projeto ligado à navegação autónoma de um robô.

Neste trabalho, criou-se um robô em CAD, numa plataforma nunca utilizada pelos elementos do grupo. Exportou-se esse robô para um ambiente de simulação, onde teve que passar por várias etapas até se obter um robô capaz de desempenhar uma navegação autónoma, através do uso do código que foi produzido e alterado de acordo com as características do robô e com o que se pretendia que ele fizesse. Por fim, ainda se adicionou mais um robô demonstrando que o código é adaptável e que se poderá futuramente ter uma série de robôs a navegar autonomamente e executando o mesmo trajeto. Uma outra sugestão consiste em testar o funcionamento destes robôs noutros cenários.

Pode-se assim afirmar que se enfrentaram algumas adversidades enquanto se desenvolvia o protótipo. Não obstante, tentou-se sempre encontrar soluções para resolver esses problemas e nos casos em que tal não foi possível, procurou-se sempre contorná-los. No final, obteve-se sucesso nos resultados alcançados visto que foram de encontro ao que era pretendido, tanto num robô como nos dois.

É de salientar ainda que todo o tempo utilizado por todos os membros do grupo foi gerido de forma a ser o mais proveitoso possível.

Por último, são projetos como estes que colocam os alunos mais próximos do mundo do trabalho, uma vez que com a realização deste projeto, se sentiu que é fulcral recorrer-se às simulações para futuramente colocar-se em prática e obter-se um bom desempenho.

6. Bibliografia

- Slides Introdução ao CoppeliaSim & Matlab
- Ficheiros para apoio aos trabalhos de simulação CoppeliaSim & Matlab
- Apresentação de apoio ao projeto P1.3 Criação de ambiente de simulação em CoppeliaSim
- Slides- An Introduction Autonomous Intelligent Systems: motivation?
- Slides Unidade Curricular de Sistemas Dinâmicos Não Lineares e Aplicações à Robótica